

P3154R0

Deprecating signed character types in iostreams

Published Proposal, 2024-02-15

This version:

<http://wg21.link/P3154R0.html>

Author:

[Elias Kosunen](#)

Audience:

LEWG, SG16

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper proposes deprecating overloads under iostreams, that take some variant of `signed char` or `unsigned char`, and treat these as characters, rather than integers. The behavior of these overloads is unexpected, especially when using the aliases `int8_t` or `uint8_t`.

Table of Contents

- 1 **Motivation**
- 2 **Impact**
- 3 **Wording**
 - 3.1 Modify [istream.general] p1
 - 3.2 Modify [istream.extractors], around p7 to p12
 - 3.3 Modify [ostream.general] p1

- 3.4 Modify [ostream.inserters.character]
- 3.5 Add a new subclause in Annex D after [depr.atomics]
- 3.6 Add a new subclause in Annex D after the above ([depr.istream.extractors])

References

Informative References

§ 1. Motivation

```
#include <iostream>
#include <format>

int main() {
    // Prints:
    std::cout
        << static_cast<          char>(48) << '\n'    // 0
        << static_cast< signed char>(48) << '\n'    // 0
        << static_cast<unsigned char>(48) << '\n'    // 0
        << static_cast<          int8_t>(48) << '\n'    // 0
        << static_cast<          uint8_t>(48) << '\n'    // 0
        << static_cast<          short>(48) << '\n'    // 48

        << std::format("{} ", static_cast<char>(48)) << '\n'    // 0
        << std::format("{} ", static_cast<int8_t>(48)) << '\n'    // 48
        << std::format("{} ", static_cast<uint8_t>(48)) << '\n'; // 48
}
```

There are overloads for `operator<<` for `basic_ostream`, that take an `(un)signed char`, and a `const (un)signed char*`. In addition, there are overloads for `operator>>` for `basic_istream`, that take an `(un)signed char&` and an `(un)signed char (&)[N]`. These overloads are specified to have equivalent behavior to the non-signedness qualified overloads: [istream.extractors] [ostream.inserters.character].

This is surprising. Per [basic.fundamental] p1 and p2:

There are five *standard signed integer types*: "signed char", "short int", "int", "long int", and "long long int"... There may also be implementation-defined *extended signed integer types*. The standard and extended signed integer types are collectively called *signed integer types*.

For each of the standard signed integer types, there exists a corresponding (but different) *standard unsigned integer type*: "unsigned char", "unsigned short int", "unsigned int", "unsigned long int", and "unsigned long long int"... Likewise, for each of the extended signed integer types, there exists a corresponding *extended unsigned integer types*. The standard and extended unsigned integer types are collectively called *unsigned integer types*.

Thus, `signed char` and `unsigned char` should be treated as integers, not as characters. This is highlighted by the fact, that `int8_t` and `uint8_t` are specified to be aliases to (un)signed integer types, which are in practice going to be `signed char` and `unsigned char`.

NOTE: The Solaris implementation is different, and defines `int8_t` to be `char` by default. This is not conformant.

`signed char` and `unsigned char` are not character types. Per [\[basic.fundamental\]](#) p11, since [\[P2314R4\]](#):

The types `char`, `wchar_t`, `char8_t`, `char16_t`, and `char32_t` are collectively called *character types*.

`signed char` and `unsigned char` are included in the set of *ordinary character types* and *narrow character types* ([\[basic.fundamental\]](#) p7), but these definitions are used for specifying alignment, padding, and *indeterminate values* ([\[basic.indet\]](#)), and are arguably not related to characters in the sense of pieces of text.

`std::format` has already taken a step in the right direction here, by treating `signed char` and `unsigned char` as integers. It's specified to not give special treatment to these types, but to use the standard definitions of (un)signed integer type to determine whether a type is to be treated as an integer when formatting.

This paper proposes that these overloads in iostreams should be deprecated.

§ 2. Impact

It's difficult to find examples where this is the sought-after behavior, and would become deprecated with this change. These snippets aren't easily greppable.

It's easy to find counter-examples, however, where workarounds have to be employed to insert or extract `signed chars` or `unsigned chars` as integers. Some of them can be found with [isocpp.org codesearch](https://isocpp.org/codesearch) by searching for `<< static_cast<int>` or `<< (int)`, although false positives there are very prevalent.

```
/* ... */ << static_cast<int>(my_schar);
```

These overloads have existed since C++98.

§ 3. Wording

This wording is relative to [\[N4971\]](#).

§ 3.1. Modify [\[istream.general\]](#) p1

```
// ...

// [istream.extractors], character extraction templates
template<class charT, class traits>
    basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>&, c
template<class traits>
basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, uns
template<class traits>
basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, sig

template<class charT, class traits, size_t N>
    basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>&, c
template<class traits, size_t N>
basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, uns
template<class traits, size_t N>
basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, sig
```

§ 3.2. Modify [istream.extractors], around p7 to p12

```
template<class charT, class traits, size_t N>
    basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>&, c
template<class traits, size_t N>
basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, uns
template<class traits, size_t N>
basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, sic
```

Effects: Behaves like a formatted input member (as described in [istream.formatted.reqmts]) of `in`. After a sentry object is constructed, `operator>>` extracts characters and stores them into `s`. If `width()` is greater than zero, `n` is `min(size_t(width()), N)`. Otherwise `n` is `N`. `n` is the maximum number of characters stored.

Characters are extracted and stored until any of the following occurs:

- `n-1` characters are stored;
- end of file occurs on the input sequence;
- letting `ct` be `use_facet<ctype<charT>>(in.getloc())`, `ct.is(ct.space, c)` is true.

`operator>>` then stores a null byte (`charT()`) in the next position, which may be the first position if no characters were extracted. `operator>>` then calls `width(0)`.

If the function extracted no characters, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

Returns: `in`.

```
template<class charT, class traits>
    basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>&, c
template<class traits>
basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, uns
template<class traits>
basic_istream<char, traits>& operator>>(basic_istream<char, traits>&, sic
```

Effects: Behaves like a formatted input member (as described in [istream.formatted.reqmts]) of `in`. A character is extracted from `in`, if one is available, and stored in `c`. Otherwise, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

Returns: `in`.

§ 3.3. Modify [ostream.general] p1

```
// ...

// [ostream.inserters.character], character inserters
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<< (basic_ostream<charT, traits>&, c
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<< (basic_ostream<charT, traits>&, c
template<class traits>
    basic_ostream<char, traits>& operator<< (basic_ostream<char, traits>&, cha

template<class traits>
basic_ostream<char, traits>& operator<< (basic_ostream<char, traits>&, sig
template<class traits>
basic_ostream<char, traits>& operator<< (basic_ostream<char, traits>&, uns

template<class traits>
    basic_ostream<char, traits>& operator<< (basic_ostream<char, traits>&, wch
template<class traits>
    basic_ostream<char, traits>& operator<< (basic_ostream<char, traits>&, cha
template<class traits>
    basic_ostream<char, traits>& operator<< (basic_ostream<char, traits>&, cha
template<class traits>
    basic_ostream<char, traits>& operator<< (basic_ostream<char, traits>&, cha
template<class traits>
    basic_ostream<wchar_t, traits>&
        operator<< (basic_ostream<wchar_t, traits>&, char8_t) = delete;
template<class traits>
    basic_ostream<wchar_t, traits>&
        operator<< (basic_ostream<wchar_t, traits>&, char16_t) = delete;
template<class traits>
    basic_ostream<wchar_t, traits>&
        operator<< (basic_ostream<wchar_t, traits>&, char32_t) = delete;
```

```

template<class charT, class traits>
    basic_ostream<charT, traits>& operator<< (basic_ostream<charT, traits>&, c
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<< (basic_ostream<charT, traits>&, c
template<class traits>
    basic_ostream<char, traits>& operator<< (basic_ostream<char, traits>&, cor

template<class traits>
— basic_ostream<char, traits>& operator<< (basic_ostream<char, traits>&, cor
template<class traits>
— basic_ostream<char, traits>& operator<< (basic_ostream<char, traits>&, cor

template<class traits>
    basic_ostream<char, traits>&
        operator<< (basic_ostream<char, traits>&, const wchar_t*) = delete;
template<class traits>
    basic_ostream<char, traits>&
        operator<< (basic_ostream<char, traits>&, const char8_t*) = delete;
template<class traits>
    basic_ostream<char, traits>&
        operator<< (basic_ostream<char, traits>&, const char16_t*) = delete;
template<class traits>
    basic_ostream<char, traits>&
        operator<< (basic_ostream<char, traits>&, const char32_t*) = delete;
template<class traits>
    basic_ostream<wchar_t, traits>&
        operator<< (basic_ostream<wchar_t, traits>&, const char8_t*) = delete;
template<class traits>
    basic_ostream<wchar_t, traits>&
        operator<< (basic_ostream<wchar_t, traits>&, const char16_t*) = delete;
template<class traits>
    basic_ostream<wchar_t, traits>&
        operator<< (basic_ostream<wchar_t, traits>&, const char32_t*) = delete;

// ...

```

§ 3.4. Modify [ostream.inserters.character]

```
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<((basic_ostream<charT, traits>&, c
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<((basic_ostream<charT, traits>&, c
// specialization
template<class traits>
    basic_ostream<char, traits>& operator<<((basic_ostream<char, traits>&, cha
// signed and unsigned
template<class traits>
basic_ostream<char, traits>& operator<<((basic_ostream<char, traits>&, sig
template<class traits>
basic_ostream<char, traits>& operator<<((basic_ostream<char, traits>&, uns
```

Effects: Behaves as a formatted output function of `out`. Constructs a character sequence `seq`. If `c` has type `char` and the character container type of the stream is not `char`, then `seq` consists of `out.widen(c)`; otherwise `seq` consists of `c`. Determines padding for `seq` as described in [ostream.formatted.reqmts]. Inserts `seq` into `out`. Calls `os.width(0)`.

Returns: `out`.

```
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<((basic_ostream<charT, traits>&, c
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<((basic_ostream<charT, traits>&, c
template<class traits>
    basic_ostream<char, traits>& operator<<((basic_ostream<char, traits>&, cor
template<class traits>
basic_ostream<char, traits>& operator<<((basic_ostream<char, traits>&, cor
template<class traits>
basic_ostream<char, traits>& operator<<((basic_ostream<char, traits>&, cor
```

Preconditions: `s` is not a null pointer.

Effects: Behaves like a formatted inserter (as described in [ostream.formatted.reqmts]) of `out`. Creates a character sequence `seq` of `n` characters starting at `s`, each widened using `out.widen()`

([basic.ios.members]), where `n` is the number that would be computed as if by:

- `traits::length(s)` for the overload where the first argument is of type `basic_ostream<charT, traits>&` and the second is of type `const charT*`, and also for the overload where the first argument is of type `basic_ostream<char, traits>&` and the second is of type `const char*`,
- `char_traits<char>::length(s)` for the overload where the first argument is of type `basic_ostream<charT, traits>&` and the second is of type `const char*` ;
- ~~`traits::length(reinterpret_cast<const char*>(s))`~~ for the other two overloads.

Determines padding for `seq` as described in [ostream.formatted.reqmts]. Inserts `seq` into `out`. Calls `width(0)`.

Returns: `out`.

§ 3.5. Add a new subclause in Annex D after [depr.atomics]

Deprecated `signed char` and `unsigned char` extraction [depr.istream.extractors]

The following function overloads are declared in addition to those specified in [istream.extractors]:

```
template<class traits>
    basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in,
template<class traits>
    basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in,
```

Effects: Behaves like a formatted input member (as described in [istream.formatted.reqmts]) of `in`. A character is extracted from `in`, if one is available, and stored in `c`. Otherwise, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

Returns: `in`.

```
template<class traits, size_t N>
    basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in,
template<class traits, size_t N>
    basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in,
```

Effects: Behaves like a formatted input member (as described in [istream.formatted.reqmts]) of `in`. After a sentry object is constructed, `operator>>` extracts characters and stores them into `s`. If `width()` is greater than zero, `n` is `min(size_t(width()), N)`. Otherwise `n` is `N`. `n` is the maximum number of characters stored.

Characters are extracted and stored until any of the following occurs:

- `n-1` characters are stored;
- end of file occurs on the input sequence;
- letting `ct` be `use_facet<ctype<charT>>(in.getloc())`, `ct.is(ct.space, c)` is true.

`operator>>` then stores a null byte (`charT()`) in the next position, which may be the first position if no characters were extracted. `operator>>` then calls `width(0)`.

If the function extracted no characters, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

Returns: `in`.

§ 3.6. Add a new subclause in Annex D after the above ([depr.istream.extractors])

Deprecated `signed char` and `unsigned char` insertion [depr ostream.inserters]

The following function overloads are declared in addition to those specified in [ostream.inserters]:

```
template<class traits>
    basic_ostream<char, traits>& operator<< (basic_ostream<char, traits>& out,
template<class traits>
    basic_ostream<char, traits>& operator<< (basic_ostream<char, traits>& out,
```

Effects: Equivalent to: `return out << static_cast<char>(c);`.

```
template<class traits>
    basic_ostream<char, traits>& operator<< (basic_ostream<char, traits>& out,
template<class traits>
    basic_ostream<char, traits>& operator<< (basic_ostream<char, traits>& out,
```

Effects: Equivalent to: `return out << reinterpret_cast<const char*>(s);`.

§ References

§ Informative References

[N4971]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 18 December 2023. URL: <https://wg21.link/n4971>

[P2314R4]

Jens Maurer. *Character sets and encodings*. 15 October 2021. URL: <https://wg21.link/p2314r4>